VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

INSTALLATION AND OPERATION GUIDE

BACHELOR'S THESIS: A DEEP LEARNING APPROACH FOR ARTWORK RECONSTRUCTION

(An extended framework based on Mask-A

Student: Ngo Thi Thuong
Student ID: ITCSIU21160
Thesis Advisor: Le Thi Ngoc Hanh, Ph.D
Submission Date: June 2026

GitHub Link: thuongngo050902/THESIS

PART I. SYSTEM OVERVIEW & REPOSITORY STRUCTURE

1. Introduction

This system restores damaged portrait artwork (scratches, cracks, missing structural details) using our proposed deep learning model based on the Mask-Aware Transformer (MAT) architecture. Unlike general-purpose image restoration methods, this model is specifically optimized for artistic portrait faces using Relative Position Bias, Transformer Adapters, and Structure Guidance to preserve brushstrokes and facial geometric alignment.

2. Key Features

* Multi-Step Pipeline: Progress seamlessly through Input & Mask -> Masked Input -> Stage 1 (Coarse/Finetune) -> Final Output (Refined/Adapter).

* Interactive Canvas & Tools: Freehand mask painting using streamlit-drawable-canvas along with direct geometric presets (Cross, Rect, Scribble).

* Mask Nudge Adjustments: Easily move, scale, and shift the mask alignment via direction buttons (Left Left, Right Right, Up Up, Down Down) to center on facial damage.

* Dual Backend Modes:

- Local: Runs inference directly on the client machine (supports cuda, cpu, or mps for Apple Silicon).

- Remote: Offloads computational workloads to a remote GPU server or Google Colab instance via a FastAPI backend API (colab_inference_api.py).

* Visual Comparator & Lightbox: Side-by-side comparative views of the Original, Stage 1, Stage 2 (Final), and original MAT Baseline outputs. Includes a zoom/pan Lightbox mode for inspecting brush strokes.

3. Directory Structure

Below is the core structure of the repository outlining the function of key folders and scripts:

THESIS/ (Root workspace folder)

```
|-- checkpoints/          # Directory storing weights (.pkl files). Download files from https://drive.google.com/drive/folders/1QHMsH1fHhLzqHFy9BsJQcNuBVLVMbxIp?usp=drive_link
|-- collab/               # Scripts/utilities for training & inference on Colab or GPU servers.
|  |-- run_phase1_server_from_drive.sh   # Automatically download data & train Phase 1.
|  |__ run_phase3_server_from_phase2.sh   # Automatically resume Phase 2 & train Phase 3.
|-- datasets/             # Data loader implementation and degradation mask generation.
|  |-- dataset_512.py    # Training data loader for 512x512 resolution.
|  |__ mask_generator_512.py  # Simulates irregular paint-loss damage masks.
|-- demo_ui/              # Support scripts for the Streamlit web application.
```

| |-- inference_adapter.py # Routes inference to local device or remote FastAPI server.

| |__ parf_compare.py # Side-by-side comparison layout visualization.

```
|-- dnnlib/                    # Low-level utilities for configuration and logging (StyleGAN2-ADA).
|-- evaluation/                # Scripts for quantitative evaluation of restoration quality.
```

| |-- eval_psnr_ssim.py # Measures Peak Signal-to-Noise Ratio & Structural Similarity.

| |__ eval_fid.py # Measures Fréchet Inception Distance (FID).

```
|-- losses/                    # Core loss functions used in network optimization.
```

| |-- loss.py # Core two-stage loss of Mask-Aware Transformer.

| |__ focal_frequency_loss.py # Focal Frequency Loss for fine paint textures.

```
|-- networks/                  # Neural network architecture definitions.
```

| |-- mat.py # Main MAT model with window-based attention layers.

| |__ structure_guidance.py # Geometric structure guidance network (Phase 3).

```
|-- torch_utils/               # Deep learning helper functions.
|-- training/                  # Core training loops and parameter configurations.
|-- dataset_tool.py            # Packages raw images into uncompressed ZIP datasets.
|-- generate_image.py          # CLI batch inference script.
|-- requirements.txt           # Python dependencies list.
|__ streamlit_app.py           # Entrypoint starting the interactive Streamlit UI.
```

PART II. SYSTEM REQUIREMENTS & ENVIRONMENT SETUP

1. Hardware Requirements

a) For Inference & Web UI Demo:

- OS: Windows 10/11, macOS Big Sur or newer, or Linux Ubuntu 18.04+.
- RAM: Minimum 16 GB.
- GPU Acceleration (Recommended):
  * Windows/Linux: CUDA-capable NVIDIA GPU (Minimum 6-8 GB VRAM).
  * MacBook: M1/M2/M3 (using Metal Performance Shaders - MPS).
  * CPU-only: Supported on all platforms (slower processing speed).

b) For Model Training:

- OS: Linux Ubuntu 20.04/22.04 LTS (Recommended).
- GPU: NVIDIA GPU with Ampere architecture or newer (RTX 3090, RTX 4090, A100, RTX A6000) with >= 16GB-24GB VRAM.

- C++ Compiler and Ninja compilation package installed to compile custom CUDA kernels.

2. Clone the Repository

```
$ git clone https://github.com/thuongngo050902/THESIS.git
$ cd THESIS
```

3. Environment Setup (Recommended: Conda)

Create environment with Python 3.8

```
$ conda create -n faceart_env python=3.8 -y
$ conda activate faceart_env
```

* Alternative (Using traditional Python venv):

(On Windows PowerShell)

```
$ python -m venv .venv
$ .venv\Scripts\Activate.ps1
```

(On Linux/macOS)

```
$ python3 -m venv .venv
$ source .venv/bin/activate
```

4. Install PyTorch

Select the command corresponding to your target hardware:

- NVIDIA GPU (CUDA 11.8 - Recommended):

```
$ pip install torch==2.1.2 torchvision==0.16.2 --index-url https://download.pytorch.org/whl/cu118
```

- Apple Silicon Macbook (MPS - M1/M2/M3):

```
$ pip install torch torchvision
```

- CPU-Only:

```
$ pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
```

5. Install Dependencies & Ninja

```
$ pip install --upgrade pip
$ pip install -r requirements.txt
```

* Ninja Compiler: ninja is required for building high-performance CUDA kernels during training.

- Conda: $ conda install -c conda-forge ninja -y

- Pip: $ pip install ninja

If running on CPU-only or macOS (MPS), if compilation fails, the code defaults automatically to CPU fallbacks.

PART III. RUNNING THE STREAMLIT WEB UI

The repository provides an interactive Streamlit Web UI (streamlit_app.py) allowing users to upload damaged portrait images, draw masks, and run multi-stage restoration.

Start the Streamlit application:

```
$ streamlit run streamlit_app.py --server.port 8501 --server.address 127.0.0.1
```

Open browser: http://127.0.0.1:8501

Configure on Advanced Controls (Sidebar Panel):

- Backend Mode:

  * Local: Run models directly on your machine.

  * Remote: Run models on a remote GPU server using FastAPI. Enter the server API URL (e.g., http://127.0.0.1:8000) in the "Remote endpoint" input box. Ideal for running on a MacBook by offloading GPU processing.

- Device:

  * Set to "cuda" if using NVIDIA GPU.

  * Set to "mps" if using Apple Silicon MacBook (M1/M2/M3) to enable GPU acceleration.

  * Set to "cpu" for standard CPU execution.

- Checkpoint Paths (Download .pkl weights from Drive: https://drive.google.com/drive/folders/1QHMsH1fHhLzqHFy9BsJQcNuBVLVMbxIp?usp=drive_link):

  * Stage 1 Coarse: checkpoints/resume_phase1_from_finetune_plus_loss.pkl

  * Final Output (Phase 3): checkpoints/network-snapshot-000072.pkl

  * Original MAT (Baseline): checkpoints/Places_512_FullData.pkl

Restoration Workflow:

- Step 1 (Input & Mask): Upload a damaged portrait image. Select position-based presets (Center, Left, Right...) and click "Generate Mask" or draw a custom mask using the freehand Drawable Canvas. Use direction buttons (Left, Right, Up, Down) to nudge the mask alignment over the damage.

- Step 2 (Masked Input): Preview the input package showing the original image overlayed with a dark mask. Click "Confirm Input Package" to lock configuration.

- Step 3 (Stage 1): Preview the coarse structure restoration from the Phase 1 model.

- Step 4 (Final Output): View the final refined portrait reconstruction from the Phase 2-3 model.

- Compare: View results side-by-side and toggle Lightbox mode to zoom (mouse scroll) and pan (drag and drop) to inspect fine brushstroke details.

FastAPI Inference Server (Remote Backend):

Start the API on the remote GPU server:

```
$ uvicorn colab_inference_api:app --host 0.0.0.0 --port 8000
```

Command-Line Interface (CLI) Batch Inference:

To run batch inference on a directory of images:

```
$ python generate_image.py \
      --network checkpoints/network-snapshot-000072.pkl \
      --dpath test_sets/CelebA-HQ/images \
      --mpath test_sets/CelebA-HQ/masks \
      --outdir samples_output
```

PART IV. MODEL RETRAINING WORKFLOW

The retraining process of the artwork restoration model is divided into three consecutive training stages (Phase 1, Phase 2, and Phase 3) using uncompressed ZIP datasets formatted by `dataset_tool.py`.

1. Dataset Preparation

Compress the raw images into an uncompressed ZIP format:

```
$ python dataset_tool.py --source /path/to/raw/images --dest datasets/faceart_train.zip
--width 512 --height 512
```

Prepare two files: `datasets/faceart_train.zip` (training) and `datasets/faceart_val.zip` (validation).

2. Three-Phase Training Flow

Phase 1: Base Finetuning

- Objective: Finetune the base MAT network on the custom portrait artwork dataset using relative biases and deterministic gate configurations.
- Core Training Command:

```
$ python train.py \
      --outdir=runs/faceart_phase1_relbias_gate \
      --data=datasets/faceart_train.zip \
      --data_val=datasets/faceart_val.zip \
      --cfg=places512 \
      --resume=checkpoints/Places_512_FullData.pkl \
      --epochs=40 \
      --batch=4 \
      --lr=5e-5 \
      --enable-rel-pos-bias=True \
      --enable-mask-bias=True \
      --enable-deterministic-latent-gate=True
```

Phase 2: Transformer Adapter Training

- Objective: Freezes core network parameters and trains lightweight adapter layers at 32x32 and 16x16

scales.

- Core Training Command:

```
$ python train.py \
      --outdir=runs/faceart_phase2_tran_adapter \
      --data=datasets/faceart_train.zip \
      --data_val=datasets/faceart_val.zip \
      --cfg=places512 \

--resume=runs/faceart_phase1_relbias_gate/00000-places512/network-snapshot-000040.pkl \
      --epochs=30 \
      --batch=4 \
      --lr=2.5e-5 \
      --enable-rel-pos-bias=True \
      --enable-mask-bias=True \
      --enable-deterministic-latent-gate=True \
      --enable-tran-adapter-32=True \
      --enable-tran-adapter-16=True
```

Phase 3: Structure Guidance & Adaptive Gate Training

- Objective: Enables geometry-guided structures and the adaptive gate module to compile the final outputs.

- Core Training Command:

```
$ python train.py \
      --outdir=runs/faceart_phase3_adaptive_structure_guidance \
      --data=datasets/faceart_train.zip \
      --data_val=datasets/faceart_val.zip \
      --cfg=places512 \

--resume=runs/faceart_phase2_tran_adapter/00000-places512/network-snapshot-000030.pkl \
      --epochs=10 \
      --batch=4 \
      --lr=5e-5 \
      --enable-rel-pos-bias=True \
      --enable-mask-bias=True \
      --enable-deterministic-latent-gate=True \
      --enable-tran-adapter-32=True \
```

```
--enable-tran-adapter-16=True \
--enable-structure-guidance=True \
--enable-structure-fuse-16=True \
--enable-structure-fuse-stage2=True \
--enable-adaptive-structure-gate=True
```

3. Training Monitoring

Check the log file output inside the snapshot folder to monitor losses:

```
$ tail -f runs/faceart_phase3_adaptive_structure_guidance/00000-places512/log.txt
```

PART V. MODEL EVALUATION METRICS

Calculate quantitative quality metrics to evaluate model restoration performance:

1. Peak Signal-to-Noise Ratio (PSNR), SSIM and L1 Loss:

```
$ python evaluation/eval_psnr_ssim.py --rec_path /path/to/reconstructed/images --gt_path /path/to/ground_truth/images
```

2. Learned Perceptual Image Patch Similarity (LPIPS):

```
$ python evaluation/eval_lpips.py --rec_path /path/to/reconstructed/images --gt_path /path/to/ground_truth/images
```

3. Fréchet Inception Distance (FID):

```
$ python evaluation/eval_fid.py --rec_path /path/to/reconstructed/images --gt_path /path/to/ground_truth/images
```

End of Installation and Operation Guide.